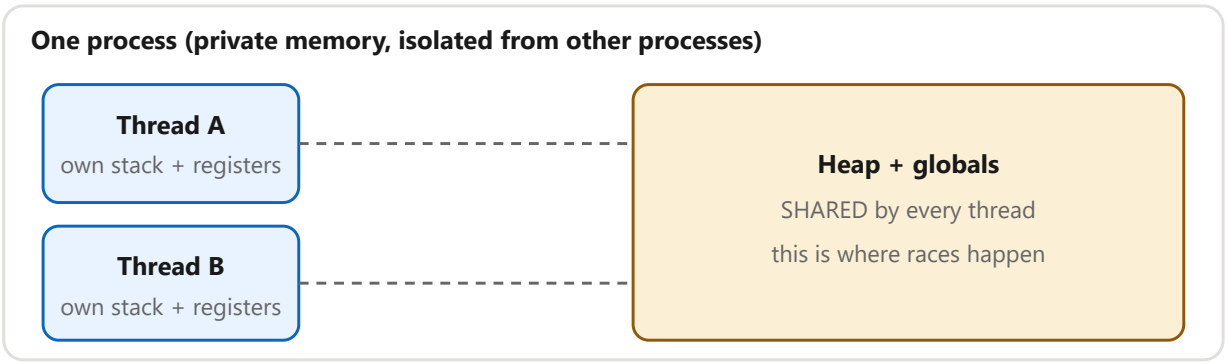
# Multithreading

From your first thread to data races, mutexes, deadlocks and condition variables: the model, the rules and the bugs. Atomics, futures and pools continue in Tasks and Parallelism and Atomics and the Memory Model.

## Why threads? The mental model




Threads share memory; that sharing is the source of both the speed and the bugs.

A **thread** is an independent flow of execution. Several threads make progress at once, like several cooks in one kitchen.

| TERM | MEANING |
|---|---|
| **Concurrency** | tasks *in progress* at overlapping times (can be interleaved on one core) |
| **Parallelism** | tasks running at the *same instant* on different cores |

**Why:** speed (use all cores), responsiveness (keep a UI or network handler alive), hiding I/O waits. **Cost:** about 1 MB stack each, startup time, context switches. Use threads only where the gain is measurable.

> THE WHOLE DIFFICULTY Threads share memory, and the OS can switch between them **between any two instructions**.

```cpp
std::thread::hardware_concurrency();     // how many hardware threads (cores x hyperthreads) this machine has, e.g. 8
```

---

## Your first thread

```cpp
#include <iostream>
#include <thread>

void hello(int id) {
    std::cout << "worker " << id << " running\n";
}

int main() {
    std::thread t(hello, 1);              // starts running hello(1) IMMEDIATELY, on a new thread
    std::cout << "main continues\n";      // main keeps going at the same time
    t.join();                             // wait here until t has finished
}
```

The **order is not fixed**. Run it repeatedly and you may see either:

```
main continues            worker 1 running
worker 1 running    or    main continues
```

A test that passed once proves almost nothing.

### Rules

- **Always `join()` or `detach()` before the `std::thread` is destroyed**, otherwise `std::terminate`. A detached thread must not touch anything that may be destroyed.
- **Arguments are copied** into the thread. Use `std::ref` to pass a reference.
- Threads are **move-only**: `std::thread b = std::move(a);`.
- An exception escaping the thread function calls `std::terminate`. Catch inside, or pass it out with a `future`.

```cpp
void addOne(int& n) { ++n; }

int x = 0;
std::thread t1(addOne, std::ref(x));      // std::ref: pass x by reference
(without it this will not even compile)
t1.join();                                // x is now 1

std::thread t2([] { std::cout << "a lambda works too\n"; });
t2.join();

struct Worker { void run(int n) { /* ... */ } } w;
std::thread t3(&Worker::run, &w, 5);      // member function: pointer to
member, the object, then the arguments
t3.join();

std::this_thread::sleep_for(std::chrono::milliseconds(100));   // pause the
current thread
std::this_thread::get_id();                                    // identify
the current thread
```

**CLASSIC DETACH BUG**

```cpp
void f() {
    int local = 0;
    std::thread t([&] { ++local; });   // captures local BY REFERENCE
    t.detach();
}                                      // f returns, local is
destroyed, the thread still writes to it
```

Capture by value (or `join()`) if the thread can outlive the variable.

CONVERTED TOO LATE

```
void f(int i, const std::string& s);

void oops() {
    char buffer[1024];
    std::thread t(f, 3, buffer);               // copies the char*
POINTER, not the text
    t.detach();                                // oops() may return
before the thread builds the std::string
}                                              // fix: std::thread t(f,
3, std::string(buffer));
```

The conversion to `std::string` happens later, inside the new thread. Convert on the calling side.

### Join on every path

If anything before `join()` can throw, the thread stays joinable and the program terminates. Use RAII:

```
class ThreadGuard {
    std::thread& t_;
public:
    explicit ThreadGuard(std::thread& t) : t_(t) {}
    ~ThreadGuard() { if (t_.joinable()) t_.join(); }     // joins however the
scope is left
    ThreadGuard(const ThreadGuard&) = delete;
    ThreadGuard& operator=(const ThreadGuard&) = delete;
};

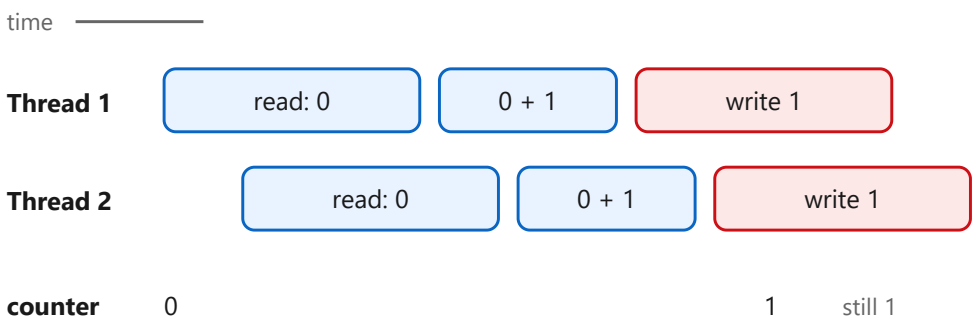
void f() {
    std::thread t(work);
    ThreadGuard g(t);                  // joins when f exits, even by an
exception
    mayThrow();
}
```

Why no implicit join or detach? A join hides long waits; a detach leaves a thread using dead locals. If a class owns a `std::thread`, declare it **last** among the members.

> **C++20: STD::JTHREAD** Joins automatically in its destructor and supports `std::stop_token` cancellation.

---

## Shared data and data races




**Expected 2, got 1: both threads read 0 before either wrote. One update is lost.**

A data race: the two read-modify-write sequences overlap. ++counter is three steps, not one.

Two threads each increment a shared counter:

```
int counter = 0;
void work() { for (int i = 0; i < 2; ++i) ++counter; }   // two threads each run this: expect counter == 4
```

`++counter` is **three** CPU steps:

1. **load** the value from memory into a register: `r = counter`
2. **add** one to the register: `r = r + 1`
3. **store** the register back to memory: `counter = r`

If B loads between A's load and store, **one increment is lost**.

| STEP | THREAD | WHAT RUNS | R IN THAT THREAD | COUNTER AFTERWARDS |
|---|---|---|---|---|
| 1 | A | `r = counter` | 0 | 0 |
| 2 | B | first increment: `r = counter`, `r = r + 1`, `counter = r` | 1 | 1 |
| 3 | B | second increment, the same three lines | 2 | 2 |
| 4 | A | `r = r + 1` (still working from the old value 0) | 1 | 2 |
| 5 | A | `counter = r` overwrites both of B's increments | 1 | 1 |
| 6 | A | second increment: `r = counter`, `r = r + 1`, `counter = r` | 2 | 2 |

One bad schedule: the final value is **2**, not 4. Two increments were lost.

> **CAVEAT** A real race may fail once in thousands of runs, which is why it survives testing.

> **DATA RACE = UNDEFINED BEHAVIOR** Two threads access the same memory, at least one writes, no synchronization. A **race condition** is the wider timing-dependent logic bug.

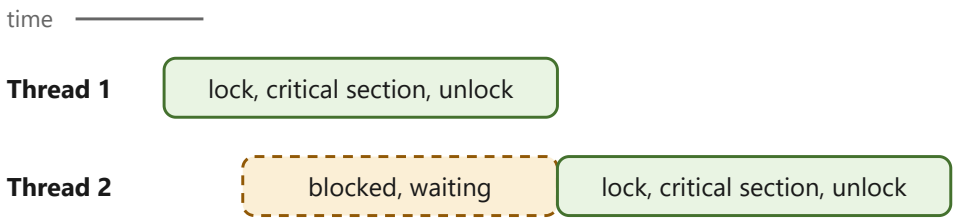
**Why it hurts:** shared data has an **invariant** (links agree, count matches items). An update breaks it briefly, and a thread looking then sees corruption. Trouble starts when *two or more* pieces of data change together.

### Three ways to be safe

| STRATEGY | HOW |
| --- | --- |
| **1. Do not share** | own data per thread (locals, `thread_local`), merge afterwards. Best option. |
| **2. Share, never modify** | `const` data is safe to read from many threads. |
| **3. Share and protect** | a **mutex** or an **atomic**. |

---

## Mutexes: taking turns




Only one thread is inside the protected region at any moment, so the updates cannot overlap.

Cost: thread 2 does nothing while it waits. Keep critical sections short.

A **mutex** lets one thread at a time into the **critical section**; others **wait** at `lock()`. Same two threads, now locked.

With the lock, the result is **always 4**.

```
std::mutex m;
int counter = 0;

void work() {
    for (int i = 0; i < 1000; ++i) {
        std::lock_guard<std::mutex> lock(m);   // locks now, unlocks
automatically at the end of the block
        ++counter;
    }
}
```

### Use RAII locks, never raw lock()/unlock()

An exception or early return between `lock()` and `unlock()` leaves the mutex locked forever.

| TYPE | USE IT WHEN |
| --- | --- |
| `std::lock_guard<std::mutex>` | default: lock for the whole scope |
| `std::unique_lock<std::mutex>` | unlock early, relock, defer; required by `cv.wait` |
| `std::scoped_lock` (C++17) | several mutexes at once, deadlock-free |
| `std::shared_mutex` + `shared_lock` | many readers or one writer |

### Protecting a class

```
class SafeCounter {
    mutable std::mutex m_;        // mutable: so const getters can still lock
it
    int value_ = 0;
public:
    void increment()   { std::lock_guard<std::mutex> lk(m_); ++value_; }
    int  get() const   { std::lock_guard<std::mutex> lk(m_); return value_; }
};
```

GOOD LOCKING

- Keep critical sections **short**; do slow work (I/O, computation) outside.
- Mutex **next to its data**; never hand out references to that data.
- Never call unknown code (callbacks, virtuals) while locked.
- Too much locking means **contention**: the program becomes serial.

## Other mutex types and `call_once`

| TYPE | WHAT IT ADDS | USE WHEN |
|---|---|---|
| `std::mutex` | nothing: one owner, not re-lockable | the default |
| `std::recursive_mutex` | the **same thread** may lock it again; it must unlock as many times | almost never: see below |
| `std::timed_mutex` | `try_lock_for(d)` and `try_lock_until(t)`: give up after a time | a thread must not wait forever (watchdogs, real-time deadlines) |
| `std::shared_mutex` (C++17) | **shared** mode for readers, **exclusive** mode for writers | reads dominate and the critical section is not tiny |

### recursive_mutex is a design smell

```
class Account {
    std::recursive_mutex m_;
    int balance_ = 0;
public:
    void deposit(int n)      { std::lock_guard lk(m_); balance_ += n; }
    void depositTwice(int n) { std::lock_guard lk(m_); deposit(n);
deposit(n); }   // locks the SAME mutex again
};
```

It works (compiled: balance 10), but it hides a design problem: `depositTwice` does not know what state `deposit` assumes. Prefer a **public** method that locks once and a **private** `depositLocked()` that assumes the lock is held. A plain `std::mutex` locked twice by one thread is undefined behaviour (usually a deadlock).

### Reader-writer lock

```
class PriceTable {
    mutable std::shared_mutex m_;
    std::map<std::string, double> px_;
public:
    void set(const std::string& k, double v) { std::unique_lock lk(m_);
px_[k] = v; }   // exclusive: one writer, no readers
    double get(const std::string& k) const {
        std::shared_lock lk(m_);
// shared: any number of readers together
        auto it = px_.find(k); return it == px_.end() ? -1 : it->second;
    }
};
```

- A `shared_mutex` costs more than a `mutex` per lock. It pays off only when there are **many readers** and each read holds the lock for a while.
- Writers can starve if readers never leave (implementation-defined fairness).
- `try_lock()` returns immediately with true or false: compiled, a second thread's `try_lock` on a held mutex returned false; `try_lock_for(20 ms)` on one held for 200 ms returned false after 20 ms.

### `std::call_once`

```
std::once_flag flag;
void init() { std::call_once(flag, [] { loadConfig(); }); }   // eight
threads may call init(): loadConfig runs exactly once
```

Use it (or a function-local `static`) for lazy initialization shared by several threads, instead of a hand-written double-checked lock.

> KEY POINT "When would you use a recursive mutex?" Rarely, and say so: it usually means a method that locks calls another that also locks. Restructure into a locking public method and a lock-assuming private one.

## Races hidden in interfaces

A mutex in every member function is necessary but **not sufficient**. Two threads run this on a shared stack whose methods all lock internally:

```
SafeStack<int> s;                  // shared by several threads; push, pop,
top, empty all lock internally

if (!s.empty()) {                  // 1. is there anything to take?
    int v = s.top();               // 2. read the top element
    s.pop();                       // 3. remove it
    use(v);
}
```

Each call is atomic, the **sequence is not**.

Two failures, neither preventable by internal locking:

- B calls `top()` on an empty stack: undefined behavior (the `empty()` answer went stale).
- Both read `top()` before either pops: one element processed **twice**.

This **check-then-act** race also hits lock-free structures. The cause is the interface, so change the interface.

### Fix: offer whole operations

```
template <typename T>
class SafeStack {
    std::stack<T> s_;
    mutable std::mutex m_;
public:
    void push(T v) {
        std::lock_guard<std::mutex> lk(m_);
        s_.push(std::move(v));
    }
    std::optional<T> try_pop() {                // check + read + remove as
ONE atomic operation
        std::lock_guard<std::mutex> lk(m_);
        if (s_.empty()) return std::nullopt;
        T v = std::move(s_.top());
        s_.pop();
        return v;
    }
    // empty() may still exist, but treat its answer as a hint only: it can
be stale the moment it returns.
};

if (auto v = s.try_pop()) use(*v);              // no window between the
check and the removal
```

**WHY DOES STD::STACK SPLIT TOP() AND POP()?** If `pop()` returned by value and the copy threw after removal, the element would be lost. A concurrent version must merge them: out-parameter, `std::optional`, `shared_ptr<T>`, or a non-throwing move.

Same pattern elsewhere:

| RACY SEQUENCE | WHOLE-OPERATION FIX |
| --- | --- |
| `if (!m.contains(k)) m[k] = v;` | one `insert` / `try_emplace` call that does both under the lock |
| `if (q.size() < max) q.push(x);` | a `try_push` that checks and pushes under one lock |
| `if (!file_exists(p)) create(p);` | an atomic "create if absent" call (for example the OS `O_EXCL` flag) |
| `if (counter.load() > 0) counter--;` | a `compare_exchange` loop |

KEY POINT "Mutex in every method, so thread-safe?" No: each call is safe, the caller's *sequence* is not. Provide combined operations like `try_pop` and protect the **whole invariant**.

---

## Deadlock

A **deadlock**: threads wait for each other forever. A holds `a` wants `b`; B holds `b` wants `a`.

| STEP | THREAD | RUNS | STATE AFTERWARDS |
| --- | --- | --- | --- |
| 1 | A | `a.lock()` | A holds a |
| 2 | B | `b.lock()` | B holds b |
| 3 | A | `b.lock()` | A waits for b, which B holds |
| 4 | B | `a.lock()` | B waits for a, which A holds: **deadlock**, neither can ever continue |

In real programs it may occur once in a million runs, only in production.

It needs all four (Coffman conditions); remove one and it cannot happen:

1. **Mutual exclusion**: one holder at a time
2. **Hold and wait**: holds one lock, waits for another
3. **No preemption**: a lock cannot be taken away
4. **Circular wait**: A waits for B, B for A

### Fix: one global lock order

If **every** thread takes `a` before `b`, no cycle is possible.

```
std::mutex a, b;

void thread1() { std::scoped_lock lk(a, b); /* ... */ }   // C++17: locks
both with a deadlock-avoidance algorithm
void thread2() { std::scoped_lock lk(b, a); /* ... */ }   // OK even though
the order differs: scoped_lock sorts it out

// pre-C++17:
std::lock(a, b);                                          // lock both
atomically (deadlock-free)
std::lock_guard<std::mutex> la(a, std::adopt_lock);       // adopt the
already-held locks so RAII still unlocks
std::lock_guard<std::mutex> lb(b, std::adopt_lock);
```

### Four guidelines, one idea

**Do not wait for a thread that might be waiting for you.**

1. **Avoid nested locks.** If you need several, take them in one step with `std::scoped_lock`.
2. **No user-supplied code under a lock.** A callback may take another lock, or keep a pointer to your data:

```
class Wrapper {
    Data data_;
    std::mutex m_;
public:
    template <typename F>
    void process(F f) {
        std::lock_guard<std::mutex> lk(m_);
        f(data_);                          // hands a reference to the
protected data to unknown code
    }
};

Data* leaked;
wrapper.process([](Data& d) { leaked = &d; });    // the callback keeps the
pointer
leaked->modify();                                  // later: modifies the
data with NO lock held
```

   Never pass out a pointer or reference to protected data.
3. **Fixed lock order.** Hand-over-hand locking on list nodes deadlocks when two threads walk in opposite directions. Fix the traversal direction.
4. **Lock hierarchy.** Each mutex has a level; a thread may only lock a *lower* level than it holds. Violations throw the first time, even if the unlucky timing never occurs. It is a small wrapper around `std::mutex` that records the current level in a `thread_local` variable and throws if you lock upward. The idea is worth knowing; in practice, `std::scoped_lock` and a documented lock order cover most programs.

NO MUTEX NEEDED Two threads that `join()` each other deadlock, as does any cycle of futures or condition-variable waits.

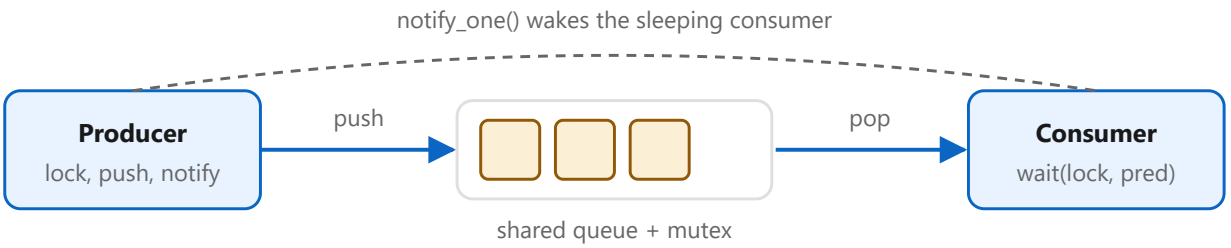
```
std::thread tA, tB;                    // both threads are started
elsewhere, running funcA and funcB

void funcA() { tB.join(); }            // A waits for B to finish ...
void funcB() { tA.join(); }            //  ... and B waits for A to finish:
neither ever returns
```

KEY POINT "Prevent deadlock?" Global lock order; `std::scoped_lock`; no foreign code under a lock; short locks; no nesting; `try_lock` with back-off. Re-locking a non-recursive mutex you own is undefined behavior.

---

## Condition variables




The consumer sleeps (no CPU) until notified, then re-checks the predicate to survive spurious wakeups.

Condition variable: sleep until something changed, always paired with a mutex and a condition.

To **wait until something happens**, a busy loop burns 100% CPU. A **condition variable** lets the thread *sleep* until woken, always together with a mutex and the shared state.

```
std::mutex m;
std::condition_variable cv;
std::queue<int> q;

// CONSUMER
std::unique_lock<std::mutex> lk(m);
cv.wait(lk, [] { return !q.empty(); });     // sleep until the predicate is
true
int x = q.front(); q.pop();

// PRODUCER
{ std::lock_guard<std::mutex> lk(m); q.push(42); }
cv.notify_one();                             // wake one waiting thread
```

### Three rules to follow

1. **Always wait with a predicate.** Spurious wakeups happen, and another consumer may take the item first.
2. **Change the state under the mutex**, then notify. Otherwise a lost wakeup: the waiter sees false, you notify, it then sleeps forever.
3. The waiter needs `unique_lock`: `wait` unlocks while sleeping.

`notify_one()` wakes one waiter; `notify_all()` wakes all (shutdown, or several may proceed).

### A reusable thread-safe queue

```
template <typename T>
class SafeQueue {
    std::queue<T> q_;
    std::mutex m_;
    std::condition_variable cv_;
public:
    void push(T v) {
        { std::lock_guard<std::mutex> lk(m_); q_.push(std::move(v)); }
        cv_.notify_one();                       // notify AFTER releasing the
lock: the woken thread can take it at once
    }
    T pop() {                                   // blocks until an item exists
        std::unique_lock<std::mutex> lk(m_);
        cv_.wait(lk, [this] { return !q_.empty(); });
        T v = std::move(q_.front());
        q_.pop();
        return v;
    }
};
```

---

## Debugging and common bugs

Timing bugs vanish under a debugger or a `printf` ("Heisenbug"). Use tools:

- **ThreadSanitizer**: `-fsanitize=thread -g` finds data races and prints both accesses. Run tests with it.
- **Helgrind / DRD** (Valgrind): races and lock-order violations.
- Thread ids in logs; `gdb` `thread apply all bt` on a hang.

| SYMPTOM | LIKELY CAUSE |
| --- | --- |
| Wrong totals, rare crashes | data race |
| Hang, CPU idle | deadlock or lost wakeup |
| Hang, CPU 100% | busy-wait or livelock |
| Crash at exit | detached thread using destroyed data; joinable thread destroyed |
| Slower with more threads | lock contention, false sharing, oversubscription |
| Garbled output | several threads on `std::cout`: build the line, print once, or lock |

### Vocabulary

- **Thread-safe**: usable from several threads without external locking (`shared_ptr`'s reference count, *not* its pointee).
- `thread_local`: one copy per thread.
- **Function-local** `static` init is thread-safe since C++11 (Meyers Singleton).
- `std::call_once(flag, fn)`: initialize exactly once.

> WHERE NEXT Atomics and the memory model are in Atomics and the Memory Model, futures, thread pools and C++20 coordination tools in Tasks and Parallelism, and false sharing, contention and pinning in Performance & Memory.

Further reading: your notes in Module 3: multithreading and synchronization (with runnable examples), the modern-cpp-tutorial concurrency chapter (threads, mutexes, futures, condition variables, atomics and the memory model), and the modern-cpp-features notes on `std::thread`, `std::async`, `std::scoped_lock` and parallel algorithms.

## Further reading by section

Where each section of this lesson is explained in more depth. Page numbers are PDF viewer pages.

### Why threads? The mental model

**C++ Concurrency in Action (Williams)**

- 1.1 What is concurrency? (PDF pages 25 to 30)
- 1.2.3 When not to use concurrency (PDF pages 32 to 33)

### Your first thread

**Effective Modern C++ (Meyers)**

- Item 37, Make std::threads unjoinable on all paths (PDF pages 268 to 276)

**C++ Concurrency in Action (Williams)**

- 2.1 Basic thread management (launching, joining, detaching, waiting in exceptional circumstances, running in the background) (PDF pages 40 to 47)
- 2.2 Passing arguments to a thread function (PDF pages 47 to 50)
- 2.3 Transferring ownership of a thread; 2.5 Identifying threads (PDF pages 50 to 53 and 57 to 59)

### Shared data and data races

**C++ Concurrency in Action (Williams)**

- 3.1 Problems with sharing data between threads (invariants, race conditions) (PDF pages 60 to 63)

### Mutexes: taking turns

**Effective Modern C++ (Meyers)**

- Item 16, Make const member functions thread safe (PDF pages 121 to 127)

**C++ Concurrency in Action (Williams)**

- 3.2.1 to 3.2.3 Mutexes in C++, structuring code to protect data, race conditions inherent in interfaces (PDF pages 64 to 74)
- 3.2.6 to 3.2.8 std::unique_lock, transferring mutex ownership, locking granularity (PDF pages 82 to 87)

### Other mutex types and call_once

**C++ Concurrency in Action (Williams)**

- 3.3.1 Protecting shared data during initialization (std::call_once) (PDF pages 88 to 91)
- 3.3.2 to 3.3.3 Protecting rarely updated data (reader-writer lock); recursive locking (PDF pages 91 to 95)

### Races hidden in interfaces

**C++ Concurrency in Action (Williams)**

- 3.2.1 to 3.2.3 Mutexes in C++, structuring code to protect data, race conditions inherent in interfaces (PDF pages 64 to 74)
- 6.1 to 6.2.2 Designing for concurrency, a thread-safe stack and a queue with condition variables (PDF pages 197 to 206)

### Deadlock

**C++ Concurrency in Action (Williams)**

- 3.2.4 to 3.2.5 Deadlock: the problem, the solution and further guidelines (PDF pages 74 to 82)

### Condition variables

**Effective Modern C++ (Meyers)**

- Item 39, Consider void futures for one-shot event communication (PDF pages 280 to 289)

**C++ Concurrency in Action (Williams)**

- 4.1 Waiting for an event: condition variables and a thread-safe queue (PDF pages 96 to 104)
- 6.1 to 6.2.2 Designing for concurrency, a thread-safe stack and a queue with condition variables (PDF pages 197 to 206)

## Debugging and common bugs

### C++ Concurrency in Action (Williams)

- 11.1 Types of concurrency-related bugs, and 11.2.1 reviewing code to locate them (PDF pages 363 to 367)